AWS Three-Tier Web Application on Terraform

1. Executive summary

This project deploys a production-style three-tier web application on AWS using Terraform, with no manual console configuration. A public Application Load Balancer receives traffic and forwards it to an Auto Scaling group of EC2 instances in private subnets. Those instances talk to a private RDS MySQL database in a separate, fully isolated data subnet. An encrypted, private S3 bucket holds application assets.

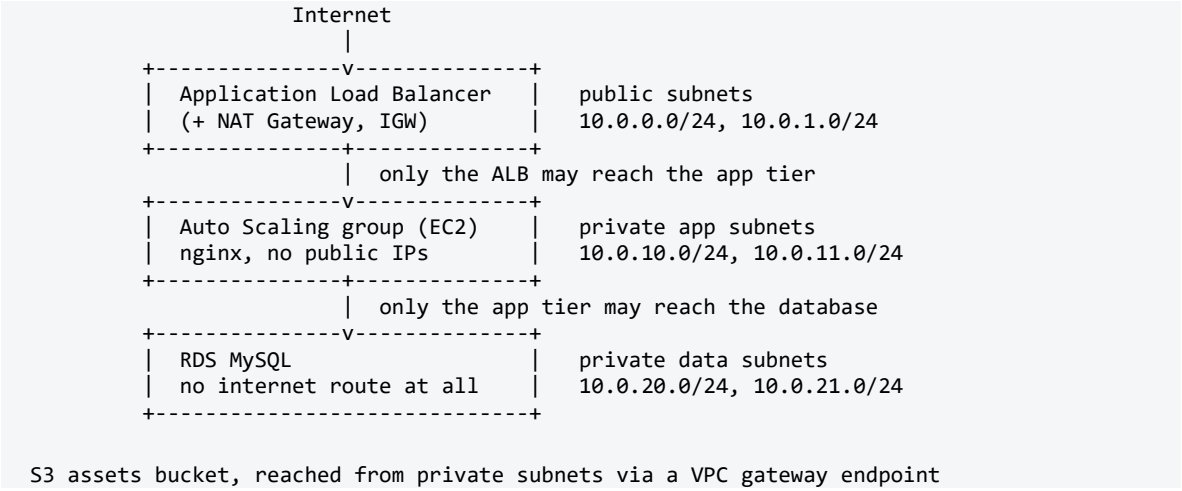
The stack was deployed successfully: all 55 resources were created, both web servers passed their load balancer health checks across two Availability Zones, and the application page was served through the load balancer's public DNS name. The entire environment can be destroyed and rebuilt from code with two commands, which keeps cost near zero between working sessions.

2. Objectives and scope

Objective	Delivered by
Isolated network with public and private subnets	VPC, 6 subnets across 2 AZs, IGW, NAT Gateway, route tables
Scalable, load-balanced web tier	Application Load Balancer, launch template, Auto Scaling group
Managed relational database not exposed to the internet	RDS MySQL in private data subnets
Object storage for assets	Private, versioned, encrypted S3 bucket
Everything reproducible from code	Terraform, 55 resources, one command to build or destroy

Out of scope for this version: HTTPS and custom domain, CI/CD pipeline, application code beyond a demo page, monitoring and alerting, multi-region failover.

3. Architecture



Each tier lives in its own pair of subnets, one per Availability Zone, so the loss of a single AZ leaves a working copy of the web tier.

4. Network design

The VPC uses `10.0.0.0/16`, divided into six `/24` subnets with `cidrsubnet()` so the layout is calculated rather than hard-coded.

Tier	Subnets	Internet path
Public	10.0.0.0/24, 10.0.1.0/24	Direct, via Internet Gateway
Private app	10.0.10.0/24, 10.0.11.0/24	Outbound only, via NAT Gateway
Private data	10.0.20.0/24, 10.0.21.0/24	None. The route table has no default route

Key choices:

- **One private route table per AZ**, so each can point at its own NAT Gateway if high availability is switched on later.
- **Single shared NAT Gateway by default** (`single_nat_gateway = true`) to cut cost. It is a single point of failure for outbound traffic from the app tier, which is acceptable for development.
- **S3 gateway endpoint** attached to the private and data route tables, so S3 traffic stays on the AWS network and avoids NAT data-processing charges.
- Availability Zones are looked up dynamically, so the same code works in any region.

5. Security design

5.1 Chained security groups

Security groups reference each other instead of IP ranges, so the tier boundaries hold even if subnet addressing changes.

Security group	Inbound allowed from	Port
ALB	Configured CIDR list (`ingress_cidrs`)	80 (and 443 when a certificate is supplied)
App tier	ALB security group only	80
Database	App-tier security group only	3306

During testing, `ingress_cidrs` was restricted to a single /32 address, so the public load balancer was reachable only from the tester's own connection.

5.2 Access and identity

- **No SSH, no bastion, no key pairs.** Instances carry the `AmazonSSMManagedInstanceCore` policy and are reached through AWS Systems Manager Session Manager.
- **Least-privilege instance role.** Beyond Session Manager, the role can read and write only the assets bucket and read exactly one secret.
- **IMDSv2 required** on every instance (`http_tokens = required`), which blocks the common SSRF-to-credentials attack path.

5.3 Data protection

- **Secrets Manager** holds the database credentials. The master password is generated by Terraform's `random_password` and never appears in a variable file or in source control.
- **Encryption at rest** on EBS volumes, RDS storage and the S3 bucket.
- **S3 hardening:** all public access blocked, bucket-owner-enforced object ownership, versioning on, and a bucket policy denying any request that is not made over TLS.
- **The database is never public** (`publicly_accessible = false`) and sits in subnets with no internet route.

6. Compute tier

Setting	Value
AMI	Latest Amazon Linux 2023, resolved at plan time from the public SSM parameter
Instance type	t3.micro (variable)
Capacity	min 2, desired 2, max 4
Placement	Both private app subnets
Health checks	ELB-based, with a 120-second grace period
Scaling policy	Target tracking on average CPU, 50% target
Deployments	Rolling instance refresh, keeping at least 50% healthy
Monitoring	Detailed (1-minute) CloudWatch metrics

Each instance bootstraps itself through `user_data`: it installs nginx, writes a `/health` endpoint that returns `200` for the load balancer, and serves a page showing the instance ID, Availability Zone,

database endpoint and bucket name. The launch template depends on the secret version, so instances never start before the credentials exist.

7. Data tier

Setting	Value
Engine	MySQL 8.0 (Postgres supported through a variable)
Class and storage	db.t3.micro, 20 GB gp3, autoscaling up to 100 GB
Placement	DB subnet group spanning both data subnets
Backups	7-day retention, nightly window, maintenance window on Mondays
High availability	Multi-AZ available through `db_multi_az` (off by default)
Protection	`deletion_protection` variable; when enabled, a final snapshot is taken on destroy

Performance Insights is disabled, because it is not supported on the db.t3.micro instance class. It can be enabled again if the instance class is upgraded.

8. Storage tier

The S3 bucket receives a random suffix to guarantee a globally unique name. It is encrypted with SSE-S3 and bucket keys, versioned, and governed by a lifecycle rule that expires non-current versions after 30 days and cleans up incomplete multipart uploads after 7 days.

9. Code structure

The project is a single Terraform root module split by concern.

File	Responsibility
`versions.tf`	Terraform and provider version pinning, optional S3 remote backend
`variables.tf`	Every tunable setting with sensible defaults and validation
`vpc.tf`	VPC, subnets, gateways, route tables, S3 endpoint
`security_groups.tf`	The three chained security groups
`alb.tf`	Load balancer, target group, HTTP and HTTPS listeners
`compute.tf`	IAM role, launch template, Auto Scaling group and policy
`rds.tf`	Subnet group, generated password, secret, DB instance
`s3.tf`	Hardened assets bucket
`outputs.tf`	Application URL, DB endpoint, secret ARN, subnet and VPC IDs
`templates/user_data.sh`	Instance bootstrap script

File	Responsibility
`terraform.tfvars.example`	Starting values to copy and edit

The module stays compact and reusable through a few Terraform patterns: `count` with `cidrsubnet()` derives every subnet from a single CIDR input, `dynamic` blocks switch the HTTP listener between forwarding and redirecting depending on whether a certificate is supplied, and `create_before_destroy` lifecycle rules allow the launch template, target group and Auto Scaling group to be replaced without downtime.

10. Deployment workflow

The stack is deployed from a dedicated IAM user with the standard Terraform workflow. `terraform init` downloads the providers, and `terraform plan` previews all 55 resources so the region, the load balancer ingress rule and the database's public-access setting can be reviewed before anything is created. `terraform apply` then builds the environment in 12 to 15 minutes, with the database as the slowest resource at roughly 10 minutes. Personal settings live in `terraform.tfvars`, which is kept out of version control together with the state file.

11. Verification results

Check	Result
Resources created	55 added, 0 changed, 0 destroyed
Subnets	6 created across us-east-1a and us-east-1b
ALB target health	Both instances reported **healthy**
End-to-end request	The application page loaded through the ALB's public DNS name, served from an instance in us-east-1a
Database exposure	`publicly_accessible = false`, no route to the internet
Ingress restriction	ALB security group limited to the tester's /32 address

12. Cost

Left running, the default configuration costs roughly $2 to $3 per day hbi(about $60 to $80 per month), mostly the NAT Gateway and the load balancer, which cannot be paused. The environment is therefore built on demand and removed with `terraform destroy` after use, then recreated from the same code in 12 to 15 minutes. A single shared NAT Gateway and an S3 gateway endpoint further reduce the bill.

13. Limitations and next steps

The demo page verifies the web tier and load balancer but does not yet run queries against the database. Planned improvements are remote state with locking, HTTPS through ACM, and CloudWatch alarms.